

Data Analytics in Cloud Computing

Exercise 10

Azure Cognitive Services Lab 2



Task 1 – Create your first Azure Cognitive Services text-to-speech application

Introduction

Suppose you are a developer working on a team that creates narrated videos for your company's marketing department. Your company usually hires vocal talent to read video scripts, but the cost of multiple vocal takes can be expensive. Coordinating recording sessions with vocal talent and studio resources also requires a great deal of scheduling.

You have heard that Microsoft Azure Cognitive Services provides developers with APIs for creating applications that use Azure's text-to-speech features. Your manager has asked you to research how Azure Cognitive Services text-to-speech features could be used to create an application for generating narration while videos are still in progress. This could help reduce operating costs and scheduling issues.

In this task, you will learn how to use the **Azure Speech service** to create a text-to-speech application that uses both plain text and **Speech Synthesis Markup Language**, also known as **SSML**, to create audio files.

Create a Text-to-Speech Application Using a Text File

In the previous lab, you learned how to create an **Azure Computer Vision Service** resource.

Now, you first need to create a **Speech service** resource in a similar way.

Use the following options when creating the resource.

Create Speech Services ...

[Basics](#)
[Network](#)
[Identity](#)
[Tags](#)
[Review + create](#)

Transcribe audible speech into readable, searchable text. Add real-time speech translations to your apps and services. Convert text to audio nearly in real time. Quickly build speech-enabled apps and services using the programming languages you already work with. Customize speech systems to optimize quality for specific scenarios.

[Learn more](#)

Project Details

Subscription * ⓘ

Resource group * ⓘ [Create new](#)

Instance Details

Region ⓘ

Name * ⓘ ✓

Pricing tier * ⓘ

[View full pricing details](#)
[Previous](#)
[Next](#)
[Review + create](#)

Now that you have an Azure Speech Service resource to work with, you can begin working on your text-to-speech application.

In this module, we're going to focus on two ways to pass text to Azure Cognitive Services' text-to-speech engine:

Text Format	Description
Plain text	Easier to produce, but inflexible if you want to customize your text-to-speech conversion.
Speech Synthesis Markup Language (SSML)	An XML format that requires a bit of learning to produce, but powerful when you want to customize your text-to-speech conversion.

Creating a text-to-speech application for text files

To create an application that will convert text to speech using Azure Cognitive Services, your application will have to accomplish all of the following tasks.

- Include the Microsoft.CognitiveServices.Speech package.
- Read text from a file.
- Create a SpeechConfig class using the API key from your Azure Cognitive Services account.
- Create a SpeechSynthesizer class using the SpeechConfig class.
- Invoke the SpeakTextAsync() method of the SpeechSynthesizer class to convert the text to speech.
- Create an AudioDataStream class using the results of the SpeakTextAsync() method.
- Invoke the SaveToWaveFileAsync() method of the AudioDataStream class to save the audio to a WAVE file.

So, let's do that, you'll create a text file that you'll use to create an audio file by using the Azure Cognitive Services speech synthesizer.

1. In the Cloud Shell, create a directory for your application, then move to your new folder.

```
mkdir text-to-speech
cd text-to-speech
```

2. Create a new .NET Core application.

```
dotnet new console
```

This command should take a few seconds to complete.

3. When your .NET Core application has been created, add the Cognitive Services package to your application.

```
dotnet add package Microsoft.CognitiveServices.Speech
```

This command should take a few seconds to complete.

Add the code for your text-to-speech application

1. In the Cloud Shell, open the Program.cs file.

```
code Program.cs
```

2. If prompted make sure to switch to classic shell and then cd to created tex-to-speech directory. Replace the existing code in Program.cs with the following using statements, which will enable the Azure Cognitive Services Speech APIs for your application.

```
using System.Text;
using Microsoft.CognitiveServices.Speech;
using Microsoft.CognitiveServices.Speech.Audio;
```

3. Below the using statements, add the following code, which will use Azure Cognitive Services Speech APIs to convert the contents of the text file that you created earlier to create a WAVE file with the synthesized voice.

```
string azureKey = "ENTER YOUR KEY COGNITIVE SERVICES KEY FROM THE FIRST TASK";
string azureLocation = "ENTER YOUR COGNITIVE SERVICES LOCATION FROM THE FIRST TASK";
string textFile = "Shakespeare.txt";
string waveFile = "Shakespeare.wav";

try
{
    FileInfo fileInfo = new FileInfo(textFile);
    if (fileInfo.Exists)
    {
        string textContent = File.ReadAllText(fileInfo.FullName);
        var speechConfig = SpeechConfig.FromSubscription(azureKey, azureLocation);
        using var speechSynthesizer = new SpeechSynthesizer(speechConfig, null);
        var speechResult = await speechSynthesizer.SpeakTextAsync(textContent);
        using var audioDataStream = AudioDataStream.FromResult(speechResult);
        await audioDataStream.SaveToWaveFileAsync(waveFile);
    }
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

This code uses your key and location to initialize a connection to Azure Cognitive Services, then it reads the contents of the text file that you created earlier, then it uses the `SpeakTextAsync()` method of the speech synthesizer to convert the text to audio, and then it uses an audio stream to save the results to an audio file.

When you've finished adding all of the code, your file should resemble the following example.

```
using System.Text;
using Microsoft.CognitiveServices.Speech;
using Microsoft.CognitiveServices.Speech.Audio;

string azureKey = "ENTER YOUR KEY COGNITIVE SERVICES KEY FROM THE FIRST TASK";
string azureLocation = "ENTER YOUR COGNITIVE SERVICES LOCATION FROM THE FIRST TASK";
string textFile = "Shakespeare.txt";
string waveFile = "Shakespeare.wav";

try
{
    FileInfo fileInfo = new FileInfo(textFile);
    if (fileInfo.Exists)
    {
        string textContent = File.ReadAllText(fileInfo.FullName);
        var speechConfig = SpeechConfig.FromSubscription(azureKey, azureLocation);
        using var speechSynthesizer = new SpeechSynthesizer(speechConfig, null);
        var speechResult = await speechSynthesizer.SpeakTextAsync(textContent);
        using var audioDataStream = AudioDataStream.FromResult(speechResult);
        await audioDataStream.SaveToWaveFileAsync(waveFile);
    }
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

Make sure that you update the values for the `azureKey` and `azureLocation` variables with your key and location from the previous task.

4. To save your changes, press Ctrl-S to save the file, and then press Ctrl-Q to exit the editor.

Create a text file for your application to read

1. In the Cloud Shell, create a new text file that your application will read.

```
code Shakespeare.txt
```

2. When the code editor appears, enter the following text.

```
The following quotes are from act 2, scene 7, of William Shakespeare's play "As You Like It."
```

```
Thou seest we are not all alone unhappy:
This wide and universal theatre
Presents more woeful pageants than the scene
Wherein we play in.
```

```
All the world's a stage,
And all the men and women merely players:
They have their exits and their entrances;
And one man in his time plays many parts,
His acts being seven ages.
```

1. To save your changes, press Ctrl-S to save the file, and then press Ctrl-Q to exit the editor.

Run your application

2. To run your application, use the following command in the Cloud Shell.

```
dotnet run
```

3. If you don't see any errors, your application has run successfully. To verify, run the following command to get a list of files in the directory.

```
ls -l
```

You should see a response like the following example, and you should see the Shakespeare.wav in the list of files.

```
drwxr-xr-x 3 user user 4096 Oct 1 11:11 bin
drwxr-xr-x 3 user user 4096 Oct 1 11:11 obj
-rw-r--r-- 1 user user 1328 Oct 1 11:11 Program.cs
-rw-r--r-- 1 user user 413 Oct 1 11:11 Shakespeare.txt
-rw-r--r-- 1 user user 955282 Oct 1 11:11 Shakespeare.wav
-rw-r--r-- 1 user user 348 Oct 1 11:11 text-to-speech.csproj
```

Listen to your WAVE file

In order to listen to the WAVE file that your application created, you'll first need to download it. To do so, use the following steps.

1. In the Cloud Shell toolbar (top of the terminal window), click the Upload/Download files



2. This opens a small menu with:
 - Upload
 - Download
 - Manage file share
3. Click "Download"
4. After selecting "Download", a file picker dialog will open.
5. You must enter the full path to wav file including extension
6. Click "Download"
7. You will get a prompt which says "Click here to download your file", click on it.
8. Download the Shakespeare.wav file to your computer, where you can listen to it with your operating system's audio player.

Change the voice

The SpeechConfig class has a SpeechSynthesisVoiceName property that you can use to specify a voice other than the default. You can find a list of voices that you can use in the Text-to-speech section of the [Language and voice support for the Speech service](#) article.

To specify the voice, use the following steps.

1. In the Cloud Shell on the right, open the Program.cs file.

```
code Program.cs
```

2. Locate the following line of code.

```
var speechConfig = SpeechConfig.FromSubscription(azureKey, azureLocation);
```

3. Add the following line after the previous line.

```
speechConfig.SpeechSynthesisVoiceName = "en-SG-WayneNeural";
```

4. The resulting section of code should look like the following example.

```
string textContent = File.ReadAllText(fileInfo.FullName);  
var speechConfig = SpeechConfig.FromSubscription(azureKey, azureLocation);  
speechConfig.SpeechSynthesisVoiceName = "en-SG-WayneNeural";  
using var speechSynthesizer = new SpeechSynthesizer(speechConfig, null);
```

When you make this change and recompile your application, the text-to-speech synthesis will use the new voice.

You can do that by referring to the steps in previous section.

Please also use a different short text, name the output file differently and download it using the previously explained steps.

Create a text-to-speech application using an SSML file – options - examples

In the previous exercise, you learned how to create an application that read a text file, converted the contents to speech, and then created an audio file with the results. The results worked well for your company's video needs, but your manager would like to know if there was some way to exert greater control over the results of your application's text-to-speech operations. For example, your video team would like to have multiple virtual speakers in a single audio file, and they'd like to alter some of the playback parameters, such as the speaking rate and the pitch.

Fortunately for you, Azure Cognitive Services' text-to-speech features work with Speech Synthesis Markup Language (SSML), which you can use to customize the speech that your application will create.

There's a wealth of information about SSML in the [Improve synthesis with Speech Synthesis Markup Language \(SSML\)](#) article, and far too much information to include in this unit. With that in mind, this unit will contain a brief overview of SSML that will help you get started.

Creating SSML files

Starting a new SSML file

SSML files use XML syntax, and the root element for an SSML file is a <speech> element. When you add that element to a new file, it should look like the following excerpt.

```
<speech xmlns="http://www.w3.org/2001/10/synthesis" version="1.0" xml:lang="en-US">  
  
</speech>
```

Specifying different voices

To specify different voices, you would wrap text within different <voice> elements that are contained with the parent <speak> element, and you would specify the voice to use by adding a name attribute to the <voice> element, which might resemble the following excerpt.

```
<speak xmlns="http://www.w3.org/2001/10/synthesis" version="1.0" xml:lang="en-US">
  <voice name="en-AU-NatashaNeural">
    Hello world!
    This is Natasha!
  </voice>
  <voice name="en-IN-NeerjaNeural">
    Hello world!
    This is Neerja!
  </voice>
  <voice name="en-CA-LiamNeural">
    Hello world!
    This is Liam!
  </voice>
  <voice name="en-US-JennyNeural">
    Hello world!
    This is Jenny!
  </voice>
</speak>
```

Adjusting the speaking rate and pitch

To specify a different speaking rate or pitch, you would enclose text within a <prosody> element, and you would specify the rate or pitch by those respective attributes, as demonstrated in the following example.

```
<speak xmlns="http://www.w3.org/2001/10/synthesis" version="1.0" xml:lang="en-US">
  <voice name="en-AU-NatashaNeural">
    <prosody rate="-5" pitch="5%">
      Hello world!
      This is Natasha!
    </prosody>
  </voice>
  <voice name="en-IN-NeerjaNeural">
    <prosody rate="-10%" pitch="0%">
      Hello world!
      This is Neerja!
    </prosody>
  </voice>
  <voice name="en-CA-LiamNeural">
    <prosody rate="-10%" pitch="-5%">
      Hello world!
      This is Liam!
    </prosody>
  </voice>
  <voice name="en-US-JennyNeural">
    <prosody rate="-5%" pitch="0%">
      Hello world!
      This is Jenny!
    </prosody>
  </voice>
</speak>
```

Adding pauses in the speech

The <break> element is excellent for controlling when a voice should speak, which you can use to adjust the overall timing of your narration. The <break> element has a time attribute, which you use to specify the amount of delay, as shown in the following example.

```
<speak xmlns="http://www.w3.org/2001/10/synthesis" version="1.0" xml:lang="en-US">
  <voice name="en-AU-NatashaNeural">
    Hello world!
    <break time="1s" />
    This is Natasha!
  </voice>
  <voice name="en-IN-NeerjaNeural">
    <break time="1s" />
    Hello world!
    This is Neerja!
  </voice>
  <voice name="en-CA-LiamNeural">
    Hello world!
    This is Liam!
    <break time="1s" />
  </voice>
  <voice name="en-US-JennyNeural">
    <prosody rate="-5%" pitch="0%">
      Hello world!
      <break time="1s" />
      This is Jenny!
    </prosody>
  </voice>
</speak>
```

Helping the speech synthesis engine with unfamiliar words

If your application requires words that Azure Cognitive Services' text-to-speech features have a difficult time pronouncing, you can enclose the difficult words within a <phoneme> element, then you can use the ph attribute to spell out the pronunciation using the appropriate phonetic alphabet as specified by the alphabet attribute.

```
<speak xmlns="http://www.w3.org/2001/10/synthesis" version="1.0" xml:lang="en-US">
  <voice name="en-SG-WayneNeural">
    The Arizona State Flower is the <phoneme alphabet="sapi" ph="s aa w aa r o">Saguaro</phoneme> Cactus Blossom.
  </voice>
</speak>
```

For more information about working with phonetic alphabets, see [Speech service phonetic sets](#).

Creating a text-to-speech application for SSML files

The process to create an application that will convert SSML to speech using Azure Cognitive Services is similar to the text file that you created in the previous exercise. With that in mind, your application will have to accomplish all of the following tasks.

- Include the Microsoft.CognitiveServices.Speech package.
- Read SSML from a file.
- Create a SpeechConfig class using the API key from your Azure Cognitive Services account.
- Create a SpeechSynthesizer class using the SpeechConfig class.
- Invoke the SpeakSsmlAsync() method of the SpeechSynthesizer class to convert the SSML to speech.

- Create an `AudioDataStream` class using the results of the `SpeakSsmlAsync()` method.
- Invoke the `SaveToWaveFileAsync()` method of the `AudioDataStream` class to save the audio to a WAVE file.

In the next exercise, we'll look at all of those steps in detail.

Create a text-to-speech application using an SSML file – exercise

Create an SSML file for your application to read

1. In the Cloud Shell, create a new XML file that your application will read.

```
code Shakespeare.xml
```

2. When the code editor appears, enter the following xml.

```
<speech xmlns="http://www.w3.org/2001/10/synthesis" version="1.0" xml:lang="en-US">

</speech>
```

This code excerpt defines the root `<speech>` element that will contain a series of `<voice>` elements that specify other voices to use.

3. Within the `<speech>` element, add the following xml to define text that will be read by the `en-GB-LibbyNeural` voice.

```
<voice name="en-GB-LibbyNeural">
  <prosody rate="-10%" pitch="-5%">
    The following quotes are from act 2, scene 7, of William Shakespeare's play "As
    You Like It."
  </prosody>
</voice>
```

This code excerpt defines a `<voice>` element that specifies a different voice than the default, and wraps the text within a `<prosody>` element that slows the speaking rate by 10%, and lowers the pitch by 5%.

4. Add the following xml after the previous `<voice>` element to define text that will be read by the `en-GB-RyanNeural` voice.

```
<voice name="en-GB-RyanNeural">
  <prosody rate="-5%" pitch="-10%">
    <break time="1s" />
    All the world's a stage,
    And all the men and women merely players:
    <break time="1s" />
    They have their exits and their entrances;
    And one man in his time plays many parts,
    His acts being seven ages.
  </prosody>
</voice>
```

Like the last code excerpt, this excerpt defines a `<voice>` element that specifies a different voice and wraps the text within a `<prosody>` element that alters the speaking rate and pitch. However, this excerpt also contains `<break>` elements that add pauses between phrases.

5. Add the following xml after the previous <voice> element to define text that will be read by the en-IE-ConnorNeural voice.

```
<voice name="en-IE-ConnorNeural">
  <prosody rate="-10%" pitch="-5%">
    <break time="1s" />
    Thou <phoneme alphabet="sapi" ph="s i eh s t">seest</phoneme> we are not all
    alone unhappy:
    <break time="1s" />
    This wide and universal theatre
    Presents more woeful pageants than the scene
    Wherein we play in.
  </prosody>
</voice>
```

Like the last two code excerpts, this excerpt defines a <voice> element that specifies a different voice and wraps the text within a <prosody> element that alters the speaking rate and pitch, and also contains <break> elements that add a pause between phrases. However, this excerpt also contains a <phoneme> element that helps the text-to-speech engine to pronounce an older English word that it might not know how to pronounce.

6. When you've finished adding all of the code, your XML file should resemble the following example.

```
<speak xmlns="http://www.w3.org/2001/10/synthesis" version="1.0" xml:lang="en-US">

  <voice name="en-GB-LibbyNeural">
    <prosody rate="-10%" pitch="-5%">
      The following quotes are from act 2, scene 7, of William Shakespeare's play
      "As You Like It."
    </prosody>
  </voice>

  <voice name="en-GB-RyanNeural">
    <break time="1s" />
    <prosody rate="-5%" pitch="-10%">
      All the world's a stage,
      And all the men and women merely players:
    </prosody>
    <prosody rate="-10%" pitch="-5%">
      They have their exits and their entrances;
      And one man in his time plays many parts,
      His acts being seven ages.
    </prosody>
  </voice>

  <voice name="en-IE-ConnorNeural">
    <prosody rate="-10%" pitch="-5%">
      Thou <phoneme alphabet="sapi" ph="s i eh s t">seest</phoneme> we are not all
      alone unhappy:
    </prosody>
    <prosody rate="-5%" pitch="-10%">
      This wide and universal theatre
      Presents more woeful pageants than the scene
      Wherein we play in.
    </prosody>
  </voice>

</speak>
```

Update the code for your text-to-speech application for SSML

1. In the Cloud Shell, open the Program.cs file.

code Program.cs

2. Leave the existing using statements unmodified, but replace the rest with the following code, which will modify the application to use the contents of an SSML file instead of a text file.

```
string azureKey = "ENTER YOUR KEY FROM THE FIRST EXERCISE";
string azureLocation = "ENTER YOUR LOCATION FROM THE FIRST EXERCISE";
string ssmlFile = "Shakespeare.xml";
string waveFile = "Shakespeare2.wav";

try
{
    FileInfo fileInfo = new FileInfo(ssmlFile);
    if (fileInfo.Exists)
    {
        string ssmlContent = File.ReadAllText(fileInfo.FullName);
        var speechConfig = SpeechConfig.FromSubscription(azureKey,
azureLocation);
        using var speechSynthesizer = new SpeechSynthesizer(speechConfig, null);
        var speechResult = await speechSynthesizer.SpeakSsmlAsync(ssmlContent);
        using var audioDataStream = AudioDataStream.FromResult(speechResult);
        await audioDataStream.SaveToWaveFileAsync(waveFile);
    }
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

This code excerpt is largely the same as the code from the previous exercise. The main differences are the lines that specify the XML file instead of the text file, and using the `SpeakSsmlAsync()` method to process the SSML code.

3. When you've finished modifying the code, your file should resemble the following example.

```
using System.Text;
using Microsoft.CognitiveServices.Speech;
using Microsoft.CognitiveServices.Speech.Audio;
string azureKey = "ENTER YOUR KEY FROM THE FIRST EXERCISE";
string azureLocation = "ENTER YOUR LOCATION FROM THE FIRST EXERCISE";
string ssmlFile = "Shakespeare.xml";
string waveFile = "Shakespeare2.wav";

try
{
    FileInfo fileInfo = new FileInfo(ssmlFile);
    if (fileInfo.Exists)
    {
        string ssmlContent = File.ReadAllText(fileInfo.FullName);
        var speechConfig = SpeechConfig.FromSubscription(azureKey,
azureLocation);
        using var speechSynthesizer = new SpeechSynthesizer(speechConfig, null);
        var speechResult = await speechSynthesizer.SpeakSsmlAsync(ssmlContent);
        using var audioDataStream = AudioDataStream.FromResult(speechResult);
        await audioDataStream.SaveToWaveFileAsync(waveFile);
    }
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

As with the previous exercise, make sure that you update the values for the `azureKey` and `azureLocation` variables with your key and location from the first exercise.

4. To save your changes, press Ctrl-S to save the file, and then press Ctrl-Q to exit the editor.

Run your application

1. To run your application, use the following command in the Cloud Shell.

```
dotnet run
```

2. If you don't see any errors, your application has run successfully. To verify, run the following command to get a list of files in the directory.

```
ls -l
```

You should see a response like the following example, and you should see the `Shakespeare2.wav` in the list of files.

```
drwxr-xr-x 3 user user 4096 Oct 1 11:11 bin
drwxr-xr-x 3 user user 4096 Oct 1 11:11 obj
-rw-r--r-- 1 user user 1328 Oct 1 11:11 Program.cs
-rw-r--r-- 1 user user 413 Oct 1 11:11 Shakespeare.txt
-rw-r--r-- 1 user user 978260 Oct 1 11:11 Shakespeare2.wav
-rw-r--r-- 1 user user 1117 Oct 1 11:11 Shakespeare.xml
-rw-r--r-- 1 user user 348 Oct 1 11:11 text-to-speech.csproj
```

Listen to your WAVE file

Use the steps provided in previous exercise to download and listen to the file.

Task 2 – Create your first Azure Cognitive Services text-to-speech application

Introduction

Natural language processing (NLP) is one of the most common AI problems, where software must interpret text or speech in the natural form that humans use. Part of NLP is the ability to classify text, and Microsoft Azure provides ways to classify text including sentiment, language, and custom categories defined by the user.

In this module, you'll learn how to use the Language service to classify text into custom groups.

After completing this module, you'll be able to:

- Define single and multiple label classifications
- Build a custom text classification project
- Label data, train, and deploy a model
- Submit classification tasks from your own app

Understand types of classification projects

Custom text classification assigns labels, which in the Language service is a *class* that the developer defines, to text files. For example, a video game summary might be classified as "Adventure", "Strategy", "Action" or "Sports".

Custom text classification falls into two types of projects:

- **Single label classification** - you can assign only one class to each file. Following the above example, a video game summary could only be classified as "Adventure" or "Strategy".
- **Multiple label classification** - you can assign multiple classes to each file. This type of project would allow you to classify a video game summary as "Adventure" or "Adventure and Strategy".

When creating your custom text classification project, you can specify which project you want to build.

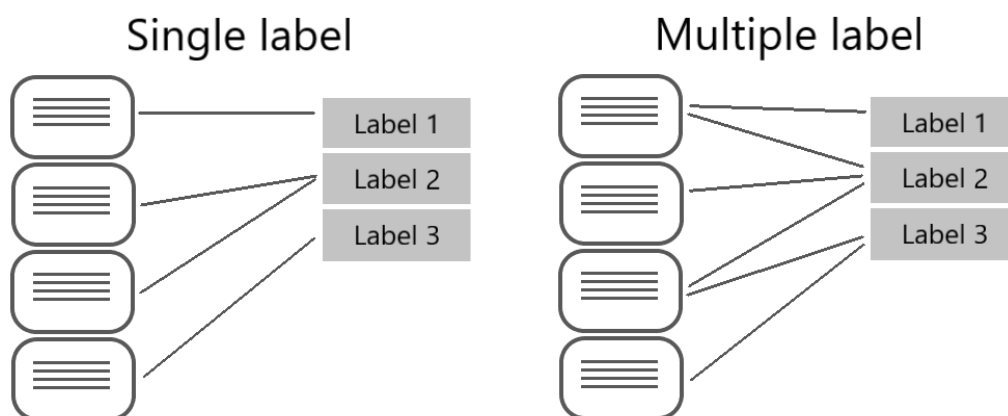
Single vs. multiple label projects

Beyond the ability to put files into multiple classifications, the key differences with multiple label classification projects are labeling, considerations for improving your model, and the API payload for classification tasks.

Labeling data

In single label projects, each file is assigned one class during the labeling process; class assignment in the Language service only allows you to select one class.

When labeling multiple label projects, you can assign as many classes that you want per file. The impact of the added complexity means your data has to remain clear and provide a good distribution of possible inputs for your model to learn from.



Labeling data correctly, especially for multiple label projects, is directly correlated with how well your model performs. The higher the quality, clarity, and variation of your data set is, the more accurate your model will be.

Evaluating and improving your model

Measuring predictive performance of your model goes beyond how many predictions were correct. Correct classifications are when the actual label is x and the model predicts a label x . In the real world, documents result in different kinds of errors when a classification isn't correct:

- False positive - model predicts x , but the file isn't labeled x
- False negative - model doesn't predict label x , but the file in fact is labeled x

These metrics are translated into three measures provided by the Language service:

- **Recall** - Of all the actual labels, how many were identified; the ratio of true positives to all that was labeled.
- **Precision** - How many of the predicted labels are correct; the ratio of true positives to all identified positives.
- **F1 Score** - A function of *recall* and *precision*, intended to provide a single score to maximize for a balance of each component

Learn more about the [Language service evaluation metrics](#), including exactly how these metrics are calculated.

With a single label project, you can identify which classes aren't classified as well as others and find more quality data to use in training your model. For multiple label projects, figuring out quality data becomes more complex due to the matrix of possible permutations of combined labels.

For example, let's say your model is correctly classifying "Action" games and some "Action and Strategy" games, but failing at "Strategy" games. To improve your model, you'll want to find more high quality and varied summaries for both "Action and Strategy" games, as well as "Strategy" games to teach your model how to differentiate the two. This challenge increases exponentially with more possible classes your model is classifying into.

API payload

The Language service provides a REST API to build and interact with your model, using a JSON body to specify the request. This API is abstracted into multiple language-specific SDKs, however for this module we'll focus our examples on the base REST API.

To submit a classification task, the API requires the JSON body to specify which task to execute. You'll learn more about the REST API in the next unit, but worth familiarizing yourself with parts of the required body.

Single label classification models specify a task of customClassificationTasks:

```
{
  "tasks": {
    "customClassificationTasks": [
      {
        "parameters": {
          "project-name": "<YOUR PROJECT>",
          "deployment-name": "<YOUR MODEL>"
        }
      }
    ]
  },
  <...>
}
```

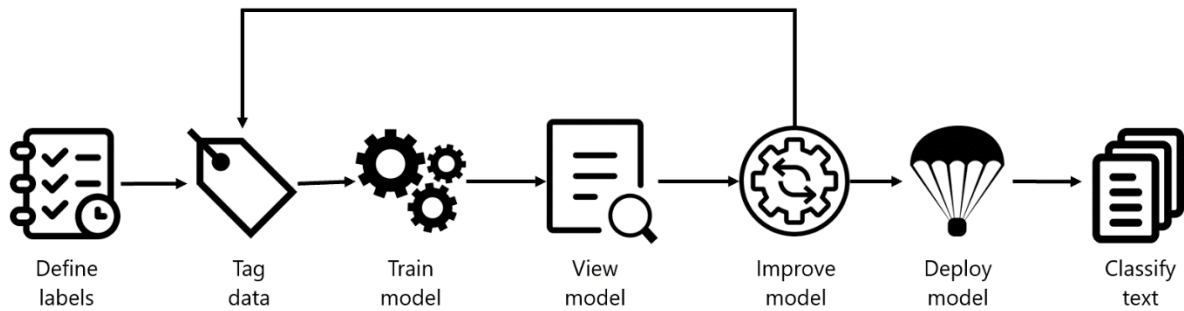
Multiple label classification models specify a task of customMultiClassificationTasks

```
{
  "tasks": {
    "customMultiClassificationTasks": [
      {
        "parameters": {
          "project-name": "<YOUR PROJECT>",
          "deployment-name": "<YOUR MODEL>"
        }
      }
    ]
  },
  <...>
}
```

Understand how to build text classification projects

Custom text classification projects are your workspace to build, train, improve, and deploy your classification model. You can work with your project in two ways: through *Language Studio* and via the REST API. Language Studio is the GUI that will be used in the lab, but the REST API has the same functionality. Regardless of which method you prefer, the steps for developing your model are the same.

Language service project life cycle



- **Define labels:** Understanding the data you want to classify, identify the possible labels you want to categorize into. In our video game example, our labels would be "Action", "Adventure", "Strategy", and so on.
- **Tag data:** Tag, or label, your existing data, specifying the label or labels each file falls under. Labeling data is important since it's how your model will learn how to classify future files. Best practice is to have clear differences between labels to avoid ambiguity, and provide good examples of each label for the model to learn from. For example, we'd label the game "Quest for the Mine Brush" as "Adventure", and "Flight Trainer" as "Action".
- **Train model:** Train your model with the labeled data. Training will teach our model what types of video game summaries should be labeled which genre.
- **View model:** After your model is trained, view the results of the model. Your model is scored between 0 and 1, based on the precision and recall of the data tested. Take note of which genre didn't perform well.
- **Improve model:** Improve your model by seeing which classifications failed to evaluate to the right label, see your label distribution, and find out what data to add to improve performance. For example, you might find your model mixes up "Adventure" and "Strategy" games. Try to find more examples of each label to add to your dataset for retraining your model.
- **Deploy model:** Once your model performs as desired, deploy your model to make it available via the API. Your model might be named "GameGenres", and once deployed can be used to classify game summaries.
- **Classify text:** Use your model for classifying text. The lab covers how to use the API, and you can view the [API reference](#).

Classify text – Exercise

Microsoft Azure Language service provides several NLP capabilities, including the key phrase identification, text summarization, and sentiment analysis. The Language service also provides custom features like custom question answering and custom text classification.

To test the custom text classification of the Language service, we'll configure the model using Language Studio then use a small command-line application that runs in the Cloud Shell to test it. The same pattern and functionality used here can be followed for real-world applications.

Create a Language service resource

To use custom text classification, you'll need to create an Azure Language service resource and select **Custom text classification & extraction** custom feature.

If you haven't already done so, create a **Language service** resource in your Azure subscription.

1. In a browser, open the [Azure portal](#), and sign in with your Microsoft account.
2. Select the **Create a resource** button, search for *Language*, and create a **Language service** resource. When asked about *Custom features*, select **Custom text classification & named entity recognition**

Home > Create a resource > Marketplace >

Select additional features

By default, Azure Cognitive Service for Language comes with several pre-built capabilities like sentiment analysis, key phrase extraction, pre-built question answering, etc. Some customizable features below require additional services like Azure Cognitive Search. [Learn more.](#)

Default features

- ✓ Sentiment analysis
- ✓ Key phrase extraction
- ✓ Pre-built question answering
- ✓ Conversational language understanding
- ✓ Named entity recognition
- ✓ Text Summarization
- ✓ Text analytics for Health

Custom features

- ✓ Custom question answering
Use this feature to answer user's questions over your data corpus. Requires Azure Cognitive Search. [Learn more.](#)
[Select](#)
- ✓ Custom text classification & Custom named entity recognition
Use this feature for custom text classification or custom named entity extraction. Requires Azure Storage. [Learn more.](#)
[Select](#)

[Continue to create your resource](#)

Create the resource with the following settings:

- **Subscription:** *Your Azure subscription.*
- **Resource group:** *Select or create a resource group with a unique name.*
- **Region:** *Choose any available region or west Europe preferably*
- **Name:** *Enter a unique name.*
- **Pricing tier:** Free F0
- **Storage account:** Create new
 - **Name:** *Enter a unique name*
 - **Storage account type:** Standard LRS
- **Responsible AI notice:** Check the box.

3. Review and create the resource.

Get Language resource key and endpoint

1. Go to the resource group in the [Azure portal](#), and select the Language resource
2. Select Keys and Endpoint from the menu on the left side, located under Resource Management. You can copy it to your clipboard with the icon next to the key. We'll need one of the keys and the endpoint later, so either paste these values into Notepad for now or we'll come back to this page at that time.

Upload sample articles

Once you've created the language service and storage account, you'll need to upload example articles to train your model later.

1. [Download sample articles](#) from this repository on GitHub. Extract the files in the .zip provided.
2. In the [Azure portal](#), navigate to the storage account you created, and select it
3. In your storage account, select Containers from the left menu, located under Data storage. On the screen that appears, select + Container. Give the container the name articles, and set Public access level to *Container (anonymous read access for containers and blobs)*. If you get message "The access level is set to private because anonymous access is disabled on this storage account" do the following to Enable anonymous access at the Storage Account level:
 - Go to the Storage Account
 - Under Settings, go to Configuration.
 - Set Allow Blob public access = Enabled.
 - Save.
4. After creating the container, select it and click the Upload button to upload the sample articles you downloaded.

Create a custom text classification project

After configuration is complete, create a custom text classification project. This project provides a working place to build, train, and deploy your model.

This lab utilizes Language Studio, but you can also create, build, train, and deploy your model through the REST API.

1. Log into the [Language Studio](#) with your Azure account, and select the Azure subscription that you created your Language resource in, and select your Language resource
2. Under the **Classify text** section, select **Custom text classification**
3. Select **Create new project**
4. In the **Create a project** pop out, choose the following and create your project:
 - **Select project type:** Single label classification
 - **Name:** ClassifyLab
 - **Description:** Custom text lab
 - **Text primary language:** English (US)
 - **Blob store container:** articles
 - **Are your files labeled with classes?:** No, I need to label my files as part of this project

If you get “The user must have "Storage Blob Data Contributor" role in the Azure Blob storage account to run Custom Text Classification” error do the following:

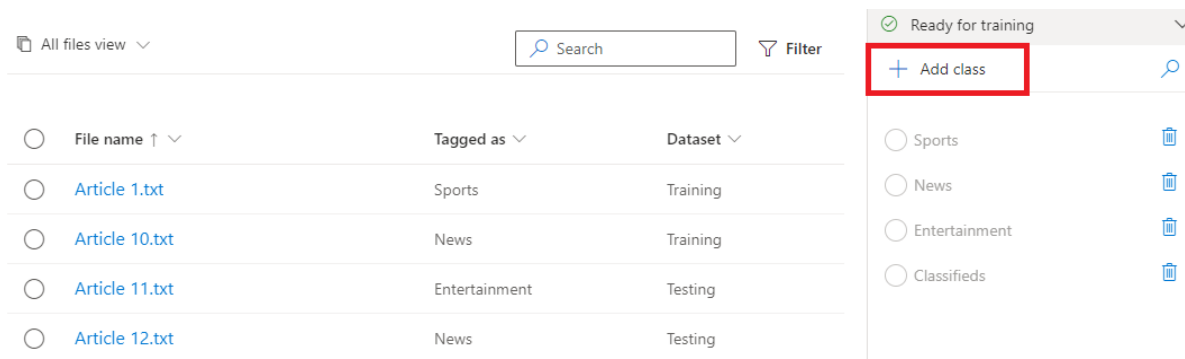
1. Go to the **Azure Portal**.
2. Navigate to your **Storage Account**.
3. On the left panel, select **Access Control (IAM)**.
4. Click **+ Add** → **Add role assignment**.
5. In the "Role" dropdown, search and select: „Storage Blob Data Contributor“
6. Under **Members**, select: **User, group, or service principal**
7. Press **+Select Members**
8. Find your user using algebra e-mail
9. Then click **select** and after **Review+assign**

After that delete the created project and create it again. If you didn't get the error, skip this step.

Label your data

Now that your project is created, you need to label, or tag, your data to train your model how to classify text.

1. On the left, click on **Data labeling**. You'll see a list of the files you uploaded to your storage account.
2. On the right side, in the **Labeling** pane, click on **Add class**. The articles in this lab fall into four classes you'll need to create: Classifieds, Sports, News, and Entertainment.



3. After you've created your four classes, start by clicking on *Article 1*. Here you can read the article, define which class this file is, and which dataset to assign it to.
4. Assign each article the appropriate class and dataset. For the purposes of this lab, we'll define which are to be used for training the model vs. testing the model.

Article	Class	Dataset
Article 1	Sports	Training
Article 10	News	Training
Article 11	Entertainment	Testing
Article 12	News	Testing
Article 13	Sports	Testing
Article 2	Sports	Training
Article 3	Classifieds	Training
Article 4	Classifieds	Training

Article 5	Entertainment	Training
Article 6	Entertainment	Training
Article 7	News	Training
Article 8	News	Training
Article 9	Entertainment	Training

Click Save labels at the end.

Train your model

Once you've labeled your data, you need to train your model.

1. Select **Training jobs** on the left side menu
2. Click **Start a training job**
3. Name your model *ClassifyArticles*
4. Choose **Use a manual split of training and testing data**
5. Click Train

In your own classification projects, the Language service will automatically split the testing set by percentage which is useful with a large dataset. With smaller datasets, it's important to train with the right class distribution.

Training your model can sometimes take several minutes. You'll get a notification when it's complete.

Evaluate your model

In real world applications of text classification, it's important to evaluate and improve your model to verify it's performing as you expect. Two pages on the left show you the details of your trained model, and any testing that failed.

Select Model performance, and select your *ClassifyArticles* model. There you can see the scoring of your model, performance metrics, and when it was trained.

Deploy your model

Once you're satisfied with the training of your model, it's time to deploy it, which allows you to start classifying text through the API.

1. On the left panel, select Deploying model
2. Select Add deployment, then enter articles and select *ClassifyArticles* to select your trained model
3. Click Submit to deploy your model
4. Once your model is deployed, leave that page open since you'll need your project and deployment name in the next step

Send text classification to your model

To test the text analytics capabilities of the Language service, we'll use a small command-line application that runs in the Cloud Shell on Azure.

Configure and run PowerShell script

Now that you have a custom model, you can run a client application that uses the Language service.

1. In the command shell, enter the following command to download the sample application and save it to a folder called ai-language.

```
git clone https://github.com/MicrosoftLearning/mslearn-ai-language ai-language
```

2. The files are downloaded to a folder named ai-language. Now we want to see all of the files in your Cloud Shell storage and work with them. Navigate to the Text Classification folder and type the following commands into the shell:

```
cd ai-language/Labfiles/04-text-classification
code classify-text.ps1
```

3. In classify-text.ps1, note the top two lines of the script with places for your Language service key and endpoint, as well as your project and model names. Replace the placeholders for \$key and \$endpoint with your resource values (\$projectName, and \$modelName should match what you entered before), and save the file.
4. Run the following command to call your model and classify the text provided. The script won't output the whole file it's classifying for the sake of space, but you can view the contents [here on GitHub](#). Review the output.

```
.\classify-text.ps1 test1.txt
```

Note: You can update the script variable \$verbose to \$true to see the raw response JSON

5. Run the following command again, this time with a different file to classify. Review the output.

```
.\classify-text.ps1 test2.txt
```

Task 3 – Recognize and convert speech to text

In this quickstart, you create and run speech resource to recognize and transcribe speech to text with it and communicate with the service using REST from your local PC.

Prerequisites:

- [Create an AI Foundry resource for Speech](#) in the Azure portal with all default options.
- Get the Speech resource key and region. After your Speech resource is deployed, select **Go to resource** to view and manage keys.
- You also need a .wav audio file on your local machine. You can use your own .wav file up to 60 seconds or download the <https://crbn.us/whatstheweatherlike.wav> sample file.

Set environment variables

You need to authenticate your application to access Azure AI services. This article shows you how to use environment variables to store your credentials. You can then access the environment variables from your code to authenticate your application. For production, use a more secure way to store and access your credentials.

To set the environment variables for your Speech resource key and region, open a console window (powershell or cmd for MS windows), and follow the instructions for your operating system and development environment.

- To set the SPEECH_KEY environment variable, replace *your-key* with one of the keys for your resource.
- To set the SPEECH_REGION environment variable, replace *your-region* with one of the regions for your resource.

```
setx SPEECH_KEY "your-key"  
setx SPEECH_REGION "your-region"
```

After you add the environment variables, you need to restart console window.

Recognize speech from a file

Open a powershell window and run the following cURL command. Replace *YourAudioFile.wav* with the path and name of your audio file.

```
curl --location --request POST  
"https://$env:SPEECH_REGION.stt.speech.microsoft.com/speech/recognition/conversat  
ion/cognitiveservices/v1?language=en-US&format=detailed" --header "Ocp-Apim-  
Subscription-Key: $env:SPEECH_KEY" --header "Content-Type: audio/wav" --data-  
binary "@YourAudioFile.wav"
```

You should receive a response similar to what is shown below. The DisplayText should be the text that was recognized from your audio file. The command recognizes up to 60 seconds of audio and converts it to text.

```
{  
  "RecognitionStatus": "Success",  
  "Offset": 700000,  
  "Duration": 12000000,  
  "DisplayText": "What's the weather like?",  
  "NBest": [  
    {  
      "Confidence": 0.952884700000000003,  
      "Lexical": "what's the weather like",  
      "ITN": "what's the weather like",  
      "MaskedITN": "what's the weather like",  
      "Display": "What's the weather like?"  
    }  
  ]  
}
```